

Experiment-6

Objective- Sequence diagrams

Sequence diagrams, commonly used by developers, model the interactions between objects in a single use case. They illustrate how the different parts of a system interact with each other to carry out a function, and the order in which the interactions occur when a particular use case is executed. In simpler words, a sequence diagram shows how different parts of a system work in a 'sequence' to get something done.

Use of Sequence diagrams

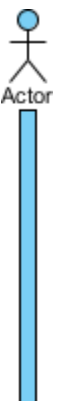
- **Model a specific use case or scenario:** Use sequence diagrams to illustrate how a system behaves during a single use case, showing the step-by-step flow of messages between objects.
- **Design or refine system architecture:** Visualize how components such as the UI, business logic, and database interact, helping architects plan or improve system structure.
- **Clarify complex processes:** Break down complicated workflows or logic into clear, time-ordered interactions, making it easier to understand and communicate system behavior.
- **Validate logic before implementation:** Confirm that all necessary interactions and message flows are defined correctly before coding begins, reducing design errors.
- **Explain system behavior to stakeholders:** Use sequence diagrams to present technical interactions in a way that both developers and non-technical team members can understand.
- **Design or analyze integrations and APIs:** Map out how services or systems exchange requests and responses to ensure smooth communication in distributed or microservice environments.

UML Sequence Diagram Symbols and Notation

A sequence diagram is structured in such a way that it represents a timeline that begins at the top and descends gradually to mark the sequence of interactions. Each object has a column and the messages exchanged between them are represented by arrows..

Actor

- a type of role played by an entity that interacts with the subject (e.g., by exchanging signals and data) external to the subject (i.e., in the sense that an instance of an actor is not a part of the instance of its corresponding subject).
- represent roles played by human users, external hardware, or other subjects.

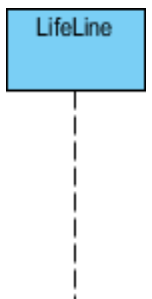


Note that:

- An actor does not necessarily represent a specific physical entity but merely a particular role of some entity
- A person may play the role of several different actors and, conversely, a given actor may be played by multiple different person.

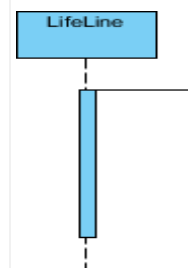
Lifeline

- A lifeline represents an individual participant in the Interaction.



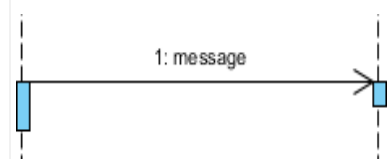
Activations

- A thin rectangle on a lifeline) represents the period during which an element is performing an operation.
- The top and the bottom of the rectangle are aligned with the initiation and the completion time respectively



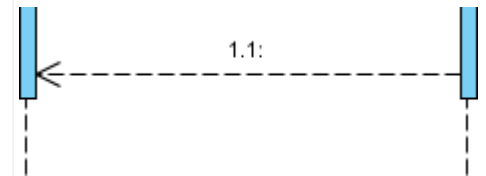
Call Message

- A message defines a particular communication between Lifelines of an Interaction.
- Call message is a kind of message that represents an invocation of operation of target lifeline.



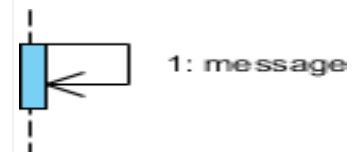
Return Message

- A message defines a particular communication between Lifelines of an Interaction.
- Return message is a kind of message that represents the pass of information back to the caller of a corresponded former message.



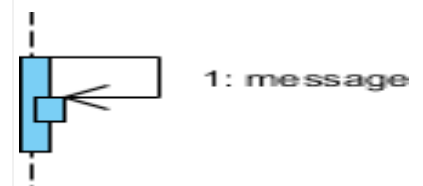
Self Message

- A message defines a particular communication between Lifelines of an Interaction.
- Self message is a kind of message that represents the invocation of message of the same lifeline.



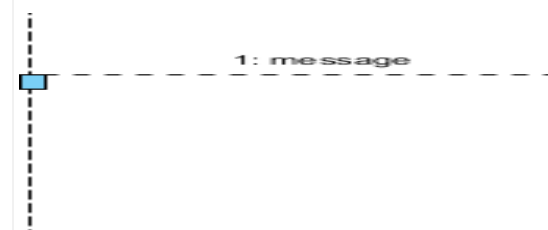
Recursive Message

- A message defines a particular communication between Lifelines of an Interaction.
- Recursive message is a kind of message that represents the invocation of message of the same lifeline. It's target points to an activation on top of the activation where the message was invoked from.



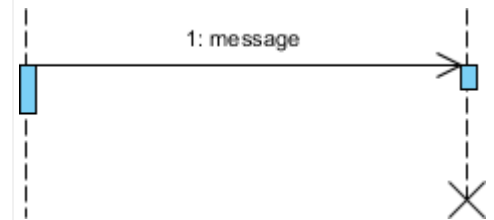
Create Message

- A message defines a particular communication between Lifelines of an Interaction.
- Create message is a kind of message that represents the instantiation of (target) lifeline.



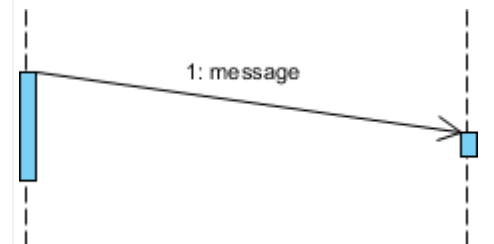
Destroy Message

- A message defines a particular communication between Lifelines of an Interaction.
- Destroy message is a kind of message that represents the request of destroying the lifecycle of target lifeline.



Duration Message

- A message defines a particular communication between Lifelines of an Interaction.
- Duration message shows the distance between two time instants for a message invocation.



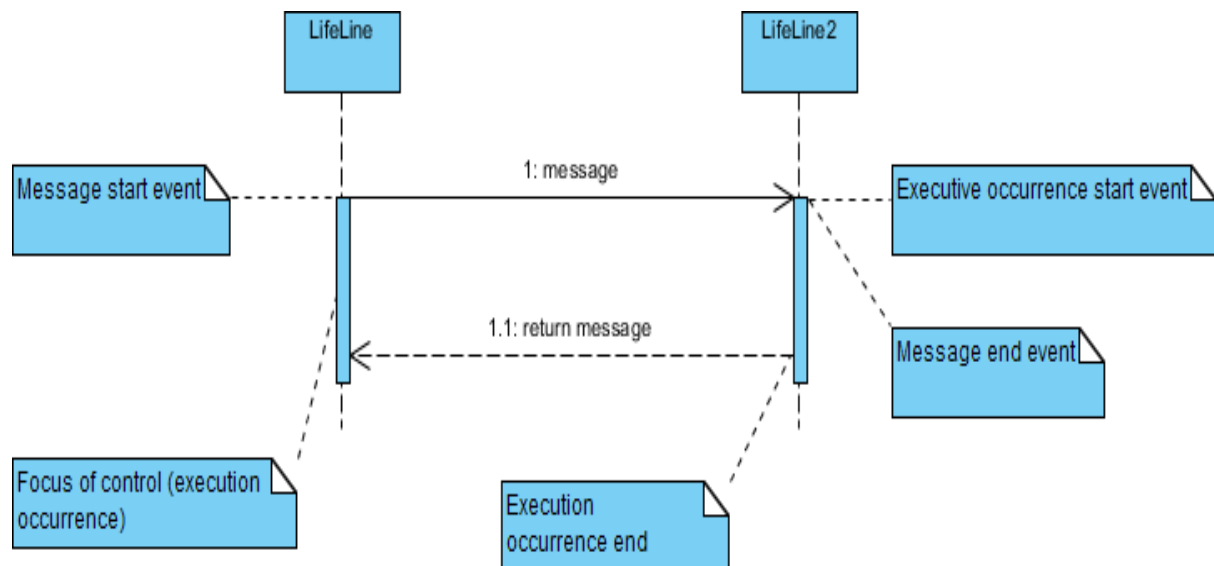
Note

A note (comment) gives the ability to attach various remarks to elements. A comment carries no semantic force, but may contain information that is useful to a modeler.



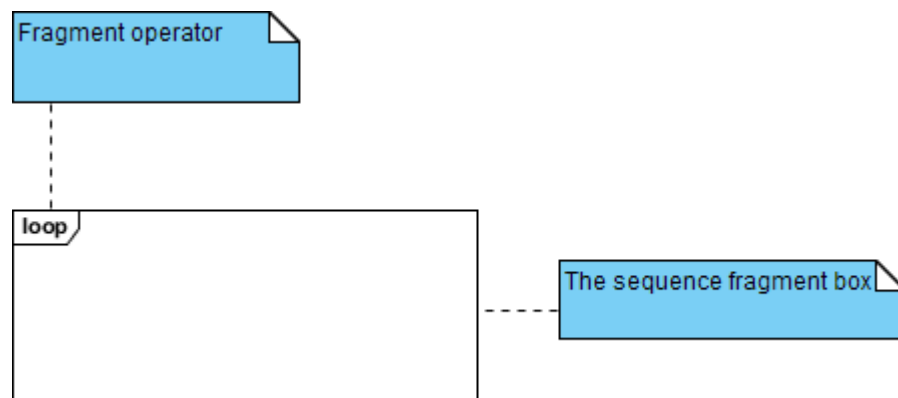
Message and Focus of Control

- An Event is any point in an interaction where something occurs.
- Focus of control: also called execution occurrence, an execution occurrence
- It shows as tall, thin rectangle on a lifeline)
- It represents the period during which an element is performing an operation. The top and the bottom of the rectangle are aligned with the initiation and the completion time respectively.



Sequence Fragments

- [UML 2.0](#) introduces sequence (or interaction) fragments. Sequence fragments make it easier to create and maintain accurate sequence diagrams
- A sequence fragment is represented as a box, called a combined fragment, which encloses a portion of the interactions within a sequence diagram
- The fragment operator (in the top left corner) indicates the type of fragment
- Fragment types: ref, assert, loop, break, alt, opt, neg



Example: Place Order

The example shows a Sequence diagram with three participating objects: Customer, Order, and the Stock. Without even knowing the notation formally, you can probably get a pretty good idea of what is going on.

- Step 1 and 2: Customer creates an order.
- Step 3: Customer add items to the order.
- Step 4, 5: Each item is checked for availability in inventory.
- Step 6, 7, 8 : If the product is available, it is added to the order.
- Step 9 return
- Step 10, 11: save and destroy order

Time

